Laravel Backend Sablon

0. Mire való ez a sablon

Ez a dokumentum Laravel backend projektekhez ad gyakorlati sablonokat: telepítés, Composer, .env, adatbázis, migration, model, controller, route, validáció, kapcsolatok, API válaszok és gyakori hibák.

1. Szükséges programok

- PHP: a Laravel futtatásához kell.
- Composer: PHP csomagkezelő, Laravel telepítéshez kell.
- MySQL vagy MariaDB: adatbázishoz.
- VS Code: kód szerkesztéséhez.
- Node.js: akkor kell, ha a Laravel mellé frontend asseteket is buildelsz Vite-tal.

Ellenőrzés terminálban

```
php -v
composer -V
node -v
npm -v
```

2. Új Laravel projekt létrehozása

```
composer create-project laravel/laravel backend-projekt
cd backend-projekt
php artisan serve
```

Várt eredmény: a projekt elindul például http://127.0.0.1:8000 címen.

> Ha a composer parancs nem működik, a Composer nincs telepítve vagy nincs PATH-ban. Telepítés után zárd be és nyisd újra a terminált.

3. .env beállítás adatbázishoz

A .env fájlban állítod be az adatbázis nevét, felhasználót és jelszót.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=turak_db
DB_USERNAME=root
DB_PASSWORD=
```

APP_KEY generálás

```
php artisan key:generate
```

4. Migration létrehozása

```
php artisan make:migration create_kategoriak_table
```

Példa migration

```
public function up(): void
{
    Schema::create('kategoriak', function (Blueprint $table) {
        $table->id();
        $table->string('nev');
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('kategoriak');
}
```

Migration futtatása

```
php artisan migrate
```

Várt eredmény: a táblák létrejönnek az adatbázisban.

5. Összetettebb migration idegen kulccsal

```
php artisan make:migration create_turak_table

public function up(): void
{
    Schema::create('turak', function (Blueprint $table) {
        $table->id();
        $table->foreignId('kategoria_id')
              ->constrained('kategoriak')
              ->cascadeOnDelete();
        $table->string('leiras');
        $table->integer('tav');
        $table->integer('emelkedes');
        $table->integer('lejtes');
        $table->boolean('aktiv')->default(true);
        $table->dateTime('indulas')->nullable();
        $table->dateTime('erkezes')->nullable();
        $table->timestamps();
    });
}
```

6. Model létrehozása

```
php artisan make:model Kategoria
php artisan make:model Tura
```

Kategoria model

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Kategoria extends Model
{
    protected $table = 'kategoriak';

    protected $fillable = [
        'nev'
    ];

    public function turak()
    {
        return $this->hasMany(Tura::class, 'kategoria_id');
```

```
    }
}
```

Tura model

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Tura extends Model
{
    protected $table = 'turak';

    protected $fillable = [
        'kategoria_id',
        'leiras',
        'tav',
        'emelkedes',
        'lejtes',
        'aktiv',
        'indulas',
        'erkezes'
    ];

    public function kategoria()
    {
        return $this->belongsTo(Kategoria::class, 'kategoria_id');
    }
}
```

7. Controller létrehozása

```
php artisan make:controller Api/TuraController
```

API controller példa

```
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Tura;
use Illuminate\Http\Request;

class TuraController extends Controller
{
    public function index()
    {
        return Tura::with('kategoria')->get();
    }

    public function show($id)
    {
        return Tura::with('kategoria')->findOrFail($id);
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'kategoria_id' => 'required|exists:kategoriak,id',
            'leiras' => 'required|string|max:255',
            'tav' => 'required|integer|min:1',
            'emelkedes' => 'required|integer|min:0',
            'lejtes' => 'required|integer|min:0',
            'aktiv' => 'required|boolean'
        ]);

        $tura = Tura::create($validated);

        return response()->json($tura, 201);
    }

    public function update(Request $request, $id)
    {
        $tura = Tura::findOrFail($id);

        $validated = $request->validate([
            'kategoria_id' => 'sometimes|exists:kategoriak,id',
```

```
            'leiras' => 'sometimes|string|max:255',
            'tav' => 'sometimes|integer|min:1',
            'emelkedes' => 'sometimes|integer|min:0',
            'lejtes' => 'sometimes|integer|min:0',
            'aktiv' => 'sometimes|boolean'
        ]);

        $tura->update($validated);

        return response()->json($tura);
    }

    public function destroy($id)
    {
        $tura = Tura::findOrFail($id);
        $tura->delete();

        return response()->json(['message' => 'Törölve']);
    }
}
```

8. API route-ok

A routes/api.php fájlba kerülnek az API útvonalak.

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\TuraController;

Route::get('/turak', [TuraController::class, 'index']);
Route::get('/turak/{id}', [TuraController::class, 'show']);
Route::post('/turak', [TuraController::class, 'store']);
Route::put('/turak/{id}', [TuraController::class, 'update']);
Route::delete('/turak/{id}', [TuraController::class, 'destroy']);
```

Route lista ellenőrzés

```
php artisan route:list
```

9. Seeder adatokkal

```
php artisan make:seeder KategoriaSeeder

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Kategoria;

class KategoriaSeeder extends Seeder
{
    public function run(): void
    {
        Kategoria::create(['nev' => 'Gyalogtúra']);
        Kategoria::create(['nev' => 'Kerékpártúra']);
        Kategoria::create(['nev' => 'Vízi túra']);
    }
}
```

DatabaseSeeder.php

```
public function run(): void
{
    $this->call([
        KategoriaSeeder::class,
    ]);
}
```

Seeder futtatás

```
php artisan db:seed
```

10. Factory próbaadatokhoz

```
php artisan make:factory TuraFactory --model=Tura
```

```
public function definition(): array
{
    return [
        'kategoria_id' => 1,
        'leiras' => fake()->sentence(),
        'tav' => fake()->numberBetween(5, 120),
        'emelkedes' => fake()->numberBetween(0, 1000),
        'lejtes' => fake()->numberBetween(0, 1000),
        'aktiv' => fake()->boolean(),
        'indulas' => now(),
        'erkezes' => now()->addHours(5),
    ];
}
```

11. Laravel API és Vue összekötése

Laravel oldali route

```
Route::get('/kategoriak', function () {
    return \App\Models\Kategoria::all();
});
```

Vue oldali hívás

```
import axios from 'axios'

const response = await axios.get('http://127.0.0.1:8000/api/kategoriak')
console.log(response.data)
```

12. Gyakori hibák és javítások

No application encryption key has been specified

```
php artisan key:generate
```

SQLSTATE Unknown database

Az adatbázis neve nincs létrehozva vagy nem egyezik a .env DB_DATABASE értékével. Hozd létre phpMyAdminban vagy MySQL-ben.

Class not found

```
composer dump-autoload
php artisan optimize:clear
```

Table already exists

Fejlesztés közben használható, de éles adatnál veszélyes:

```
php artisan migrate:fresh --seed
```

419 Page Expired

Webes POST kérésnél hiányzik a CSRF token. API útvonalhoz routes/api.php-t használj, web formhoz pedig @csrf kell.

13. Request validation külön osztályban

Nagyobb projektnél a validációt érdemes külön Request osztályba rakni.

```
php artisan make:request StoreTuraRequest
```

```
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreTuraRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'kategoria_id' => 'required|exists:kategoriak,id',
            'leiras' => 'required|string|max:255',
            'tav' => 'required|integer|min:1',
            'emelkedes' => 'required|integer|min:0',
            'lejtes' => 'required|integer|min:0',
            'aktiv' => 'required|boolean'
        ];
    }
}
```

Controllerben használva

```
use App\Http\Requests\StoreTuraRequest;

public function store(StoreTuraRequest $request)
{
    $tura = Tura::create($request->validated());
    return response()->json($tura, 201);
}
```

14. API Resource használata

```
php artisan make:resource TuraResource
```

```
namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class TuraResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'leiras' => $this->leiras,
            'tav' => $this->tav,
            'aktiv' => $this->aktiv,
            'kategoria' => [
                'id' => $this->kategoria?->id,
                'nev' => $this->kategoria?->nev,
            ]
        ];
    }
}
```

Controllerben

```
use App\Http\Resources\TuraResource;

public function index()
{
    $turak = Tura::with('kategoria')->get();
    return TuraResource::collection($turak);
}
```

15. Lapozás és keresés API-ban

```
public function index(Request $request)
{
    $query = Tura::with('kategoria');

    if ($request->filled('search')) {
        $query->where('leiras', 'like', '%' . $request->search . '%');
    }

    if ($request->filled('aktiv')) {
        $query->where('aktiv', $request->boolean('aktiv'));
    }

    return $query->orderByDesc('id')->paginate(10);
}
```

Teszt URL

```
http://127.0.0.1:8000/api/turak?search=erdei&aktiv=1
```

16. Egy-a-többhöz kapcsolat rövid minta

```
// Egy kategóriának több túrája lehet
public function turak()
{
    return $this->hasMany(Tura::class, 'kategoria_id');
}

// Egy túra egy kategóriához tartozik
public function kategoria()
{
    return $this->belongsTo(Kategoria::class, 'kategoria_id');
}
```

17. Fájl feltöltés backend oldalon

```
public function upload(Request $request)
{
    $request->validate([
        'kep' => 'required|image|max:2048'
    ]);

    $path = $request->file('kep')->store('kepek', 'public');

    return response()->json([
        'path' => $path,
        'url' => asset('storage/' . $path)
    ]);
}
```

Storage link

```
php artisan storage:link
```

18. Cache és config hibák tisztítása

```
php artisan optimize:clear
php artisan config:clear
php artisan route:clear
php artisan view:clear
composer dump-autoload
```

19. Backend ellenőrző lista

- .env adatbázis adatok jók?
- Létezik az adatbázis?
- php artisan migrate lefutott?
- Model fillable mezők be vannak állítva?
- Route szerepel a php artisan route:list listában?
- Controller namespace és use sorok jók?
- API tesztelhető böngészőből vagy Postmanből?